



AUBURN

UNIVERSITY

Processes

Objectives

- Learn and understand the concept/notion of a **Process**.
- Learn and understand how to **represent** a process.
- Learn and understand **Process Scheduling**.
- Learn and understand **Threads**.

Requirements

- Read Section 3.1
- Read Section 3.2
- Skim Section 3.3



The Concept of a Process

- **What** is a Process?
- **Life** of a Process

Process Concept

? An operating system executes a variety of programs using different strategies:

? **Batch** system – jobs

(great throughput, bad response time, poor interaction, dinosaur times)

? **Time-shared** systems – user programs or tasks

(ok throughput (less than batch), good response time, good interaction, today's systems)

? Textbook uses the terms *job* and *process* almost interchangeably

? **Process** – **a program in execution**; active entity, alive, dynamic...

What is a process?

? A **process** is a *program in execution*. A process is **alive**!

? It is **alive** as it is using a CPU, **main** memory, and I/O (files).

? It is **alive** as some of its characteristics will vary/evolve over its life:

Question: which characteristics of the process are changing?

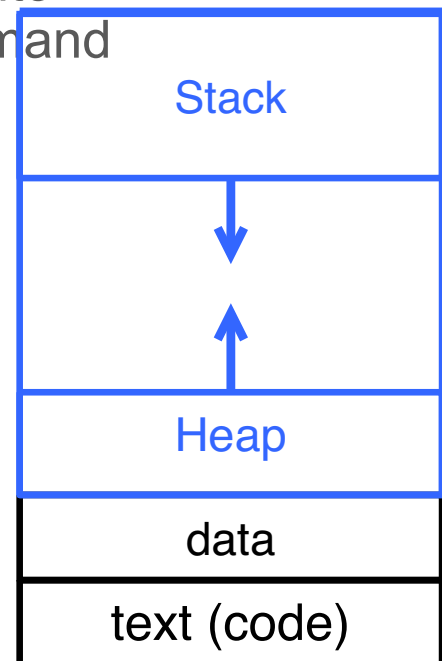
? **In contrast**, a **program** is **dead**:

? it is stored on some secondary storage. Nothing changes about it, except maybe it gets moved, destroyed....by the OS or the user.

? **Program becomes process** when executable file loaded into memory after mouse-clicking it or launching through a command prompt.

?

? Let us look at the life of a program going from death to life (becoming a process)



What is a process? (Continued)

? So, the process's stack and heap vary.... **What else will change?**

? its **next instruction to execute** (unless the code has only one machine instruction!)

? **Program Counter**

? **What it *is* doing (its *state*)**

? **new** : when it just get awoken from being a program to become a process. It is the **new** kid on the block: it does NOT yet have any resource to execute/run. It is not yet loaded in the main memory. It needs all resources (especially CPU and memory).

? **Ready**: the OS allocates it memory and upload it in the main memory. It is ready to use the CPU, but it does not have it. It is **ready** to run/execute. The process is **not yet running** because it does not “have” the CPU.

? **Running**: the OS allocates the CPU to the process. Now, the process is **running**/executing. It “has” now the CPU.

? **Waiting/Blocked**: the process initiated some I/O operations and is **waiting** them to get completed. During this time, the CPU is of no use for the process: it cannot run!

? **Terminated**: the process is done executing. It gets **terminated** by the OS to free resources (CPU, memory...)

Life Of A Process



T

Let Us Recapitulate

❓ So far, we saw that a process is **characterized** by:

```
{  
    state (new, ready, running, waiting, or terminated)  
    program counter (provides the address to the next instruction)  
    stack (keep track of the top of the stack)  
    data section  
    code  
}
```


Process Representation

- **How to represent** a Process? (in order to manage it)
- The **Process Control Block** (PCB)

Process Control Block (PCB)

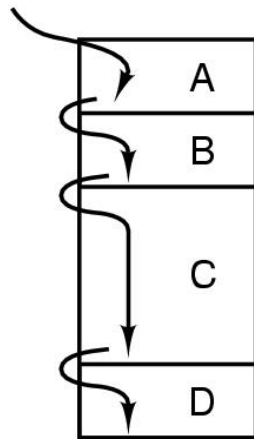
- ? A process will be represented using a data structure that contains all information that characterizes the process.
- ? This data structure is in general called a **Process Control Block**
- ? The **process control block** includes in general this information:

```
{  
    Process ID  
    Process state  
    Program counter  
    CPU registers  
    list of opened files  
    CPU scheduling information  
    Memory-management information  
    *Accounting information  
    I/O status information  
}
```



How is the PCB Used?

- ? We discuss here **multiprogramming** of four programs A, B, C, and D
- ? Only one process can be running at any time (there is only one CPU!)
- ? We must save in the memory the PCB of each process not running to know for each one what is its next instruction.
- ? Keep a “list” of PCBs of all processes currently in the system and competing for the CPU

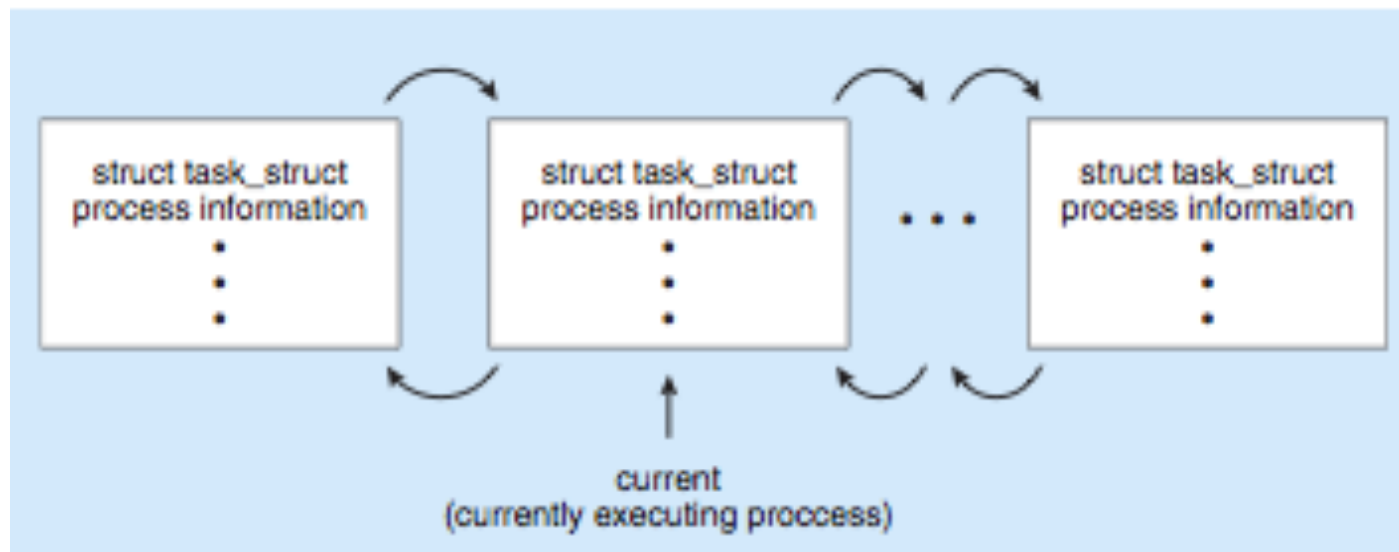


(a)

Process Representation in Linux

☐ Represented by the C structure `task_struct`

```
pid_t pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this pro */
```



Double linked list

How Does the OS switch the CPU between Processes?

? A process P_0 is running: this means that P_0 “has” the CPU.

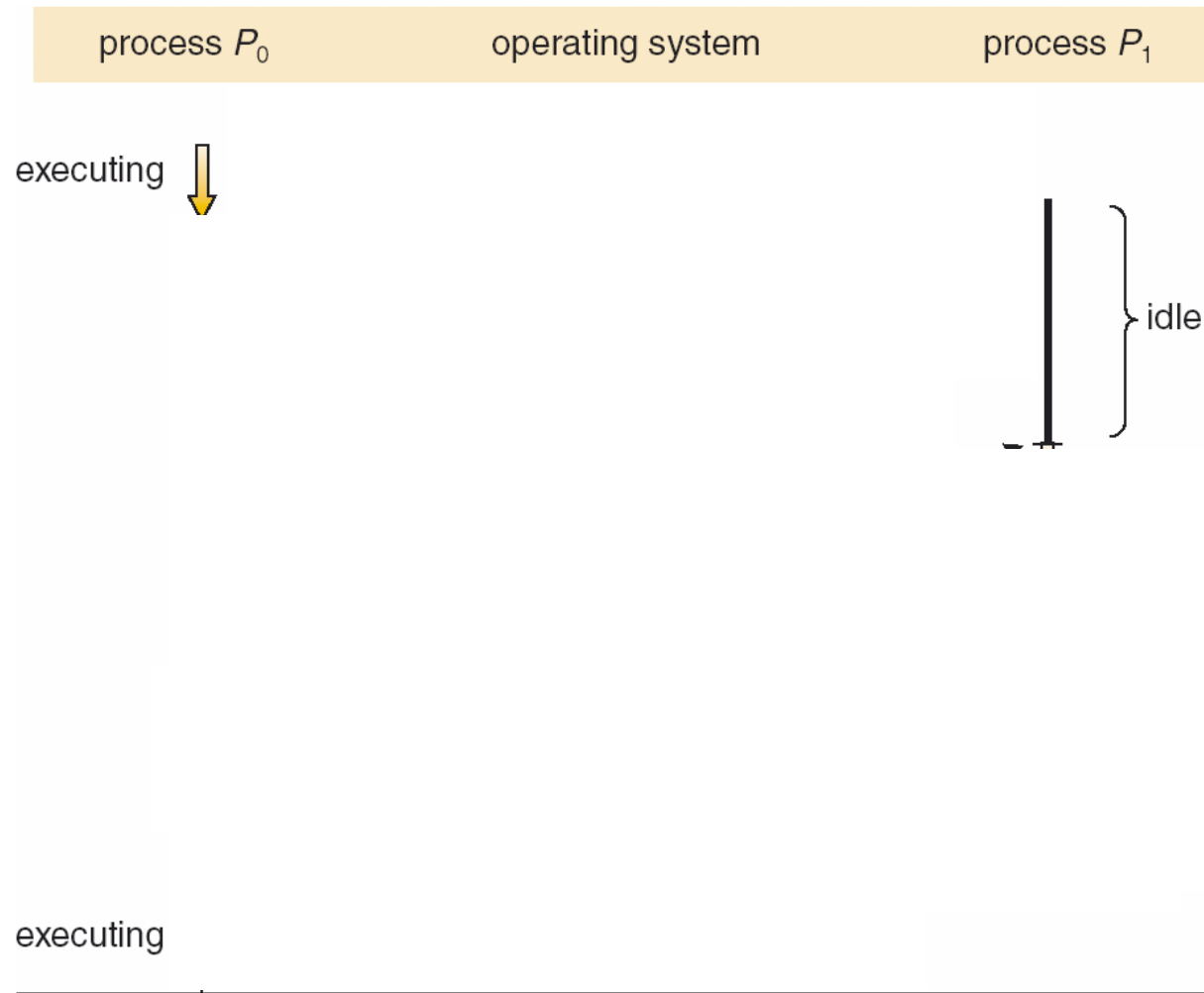
? **Question 1:** how does the OS “take back” control of the CPU if another process P_1 is currently running and not voluntarily relinquishing the CPU?

? **Answer:** Before giving control to any process, the OS “arms” a hardware timer for some duration d . After time d elapses, the timer triggers a hardware interrupt that runs a routine handler belonging to the OS: OS is back in control.

? **Question 2:** now that the OS is in control of the CPU, how does the OS complete the switch of the CPU from Process P_0 to Process P_1 .

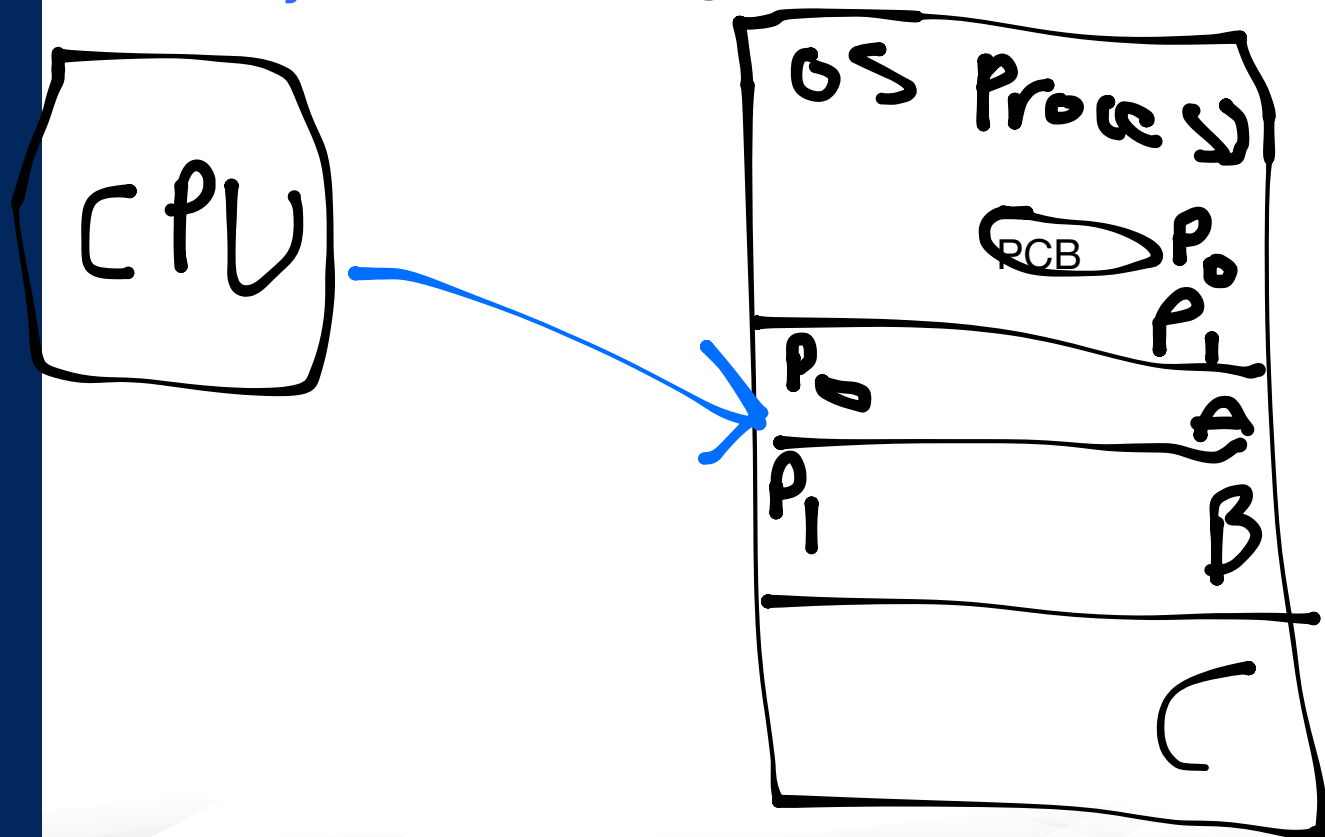
? **Answer:** this operation is called “*context switch*” or “*CPU switch*”. It is described on the next slide.

CPU Switch From Process to Process



Process Scheduling

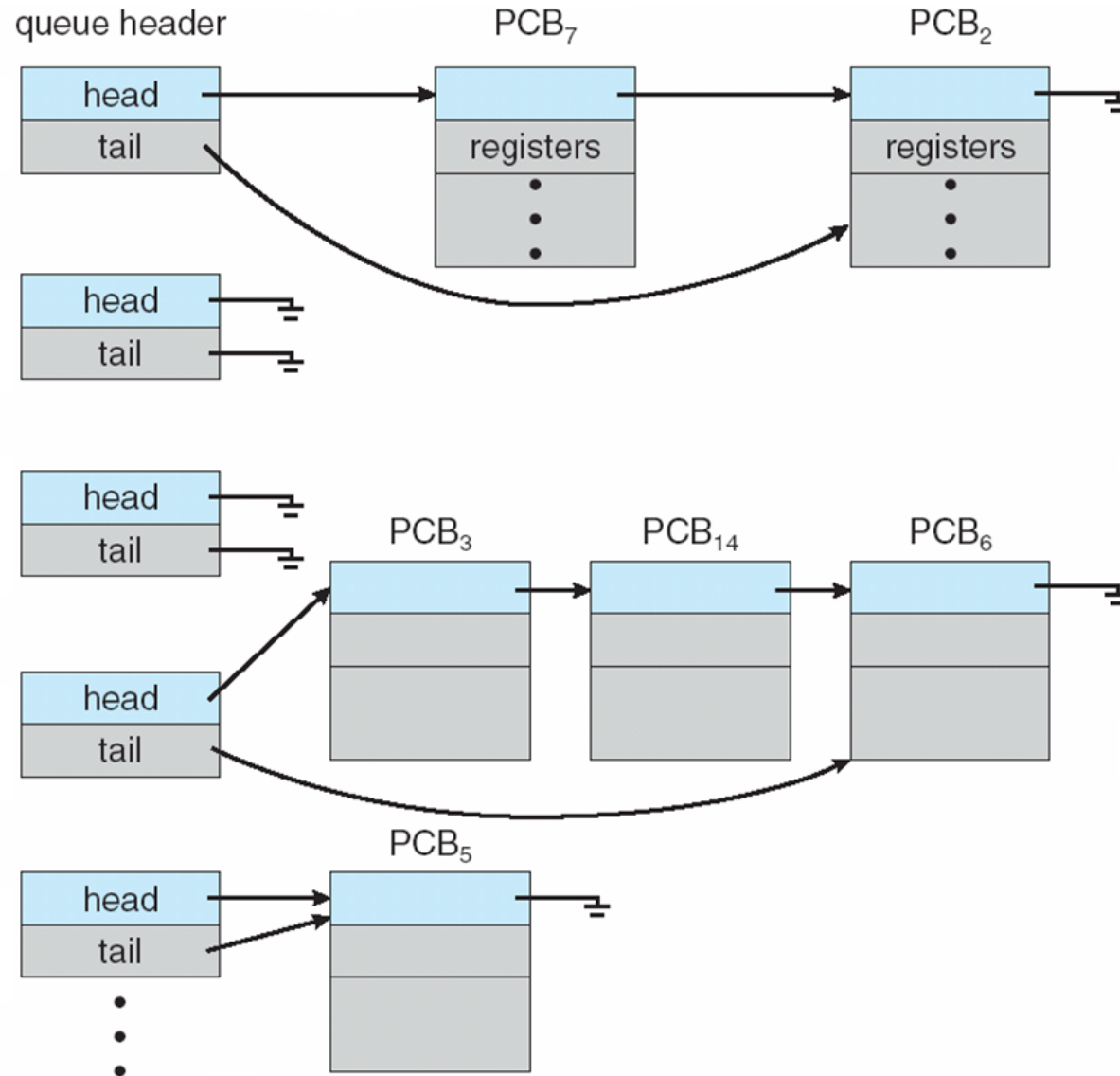
- What is **Process Scheduling**?
- **Why** Process Scheduling?



Process Scheduling

- ? **Objective:** maximize CPU use, decrease response time by quickly switch processes onto CPU for time sharing.
- ? There are different types of resources to share (CPU and resources). Therefore, we will have different schedulers.
- ? The OS maintains **different scheduling queues** of processes
 - ? **Job queue** – set of all processes just admitted in the system
 - ? **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - ? **Device queues** – set of processes waiting for an I/O device
- ? **Question:** why different queues?
 - ? **Answer:** a process may have different states. For example, if a process is waiting for an I/O operation, it is wasteful to assign it the CPU (It cannot resume before the completion of the I/O operation anyway).
- ? Processes migrate among the various queues as their states change/evolve.
- ? This task is challenging as there are two different types of processes:
 - ? **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts limited by I/O
 - ? **CPU-bound process** – spends more time doing computations; few very long CPU bursts Limited by CPU

Ready Queue And Various I/O Device Queues



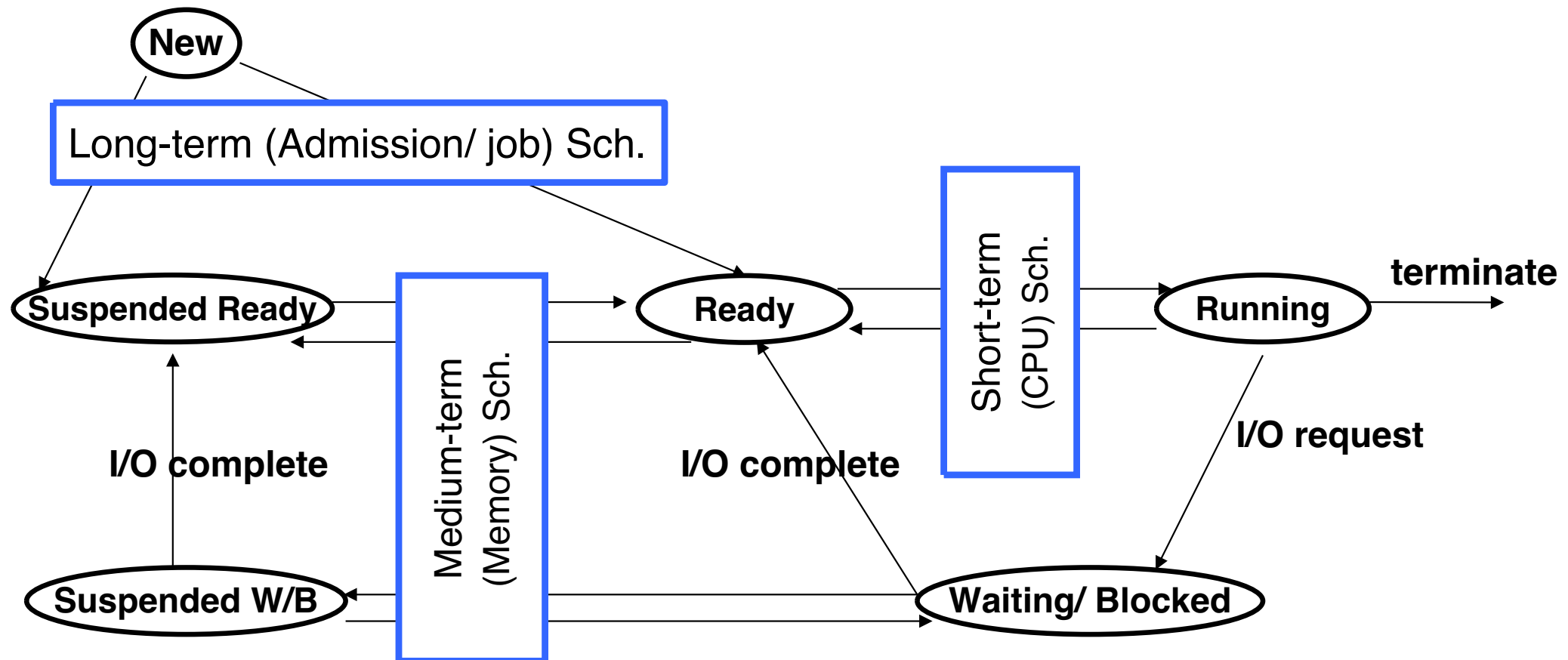
Schedulers

❓ **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue (i.e., the process can get **memory**)

❓ **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates **CPU**
manages the ready queue

❓ **Mid-term scheduler** (or **memory scheduler**) - select which process can be in **main memory** or swapped out to secondary storage (suspended)

Expanded State Transition Diagram



Schedulers (Cont.)

❑ **Short-term scheduler** is invoked very **frequently** (milliseconds) ❑
(must be fast)

❑ **Long-term scheduler** is invoked very **infrequently** (seconds, minutes) ❑ (may be slow)

❑ The long-term scheduler controls **the degree of multiprogramming** (i.e., the number of processes admitted to run concurrently in the system)

Threads

- What is **a thread**?
- **Why** use threads?

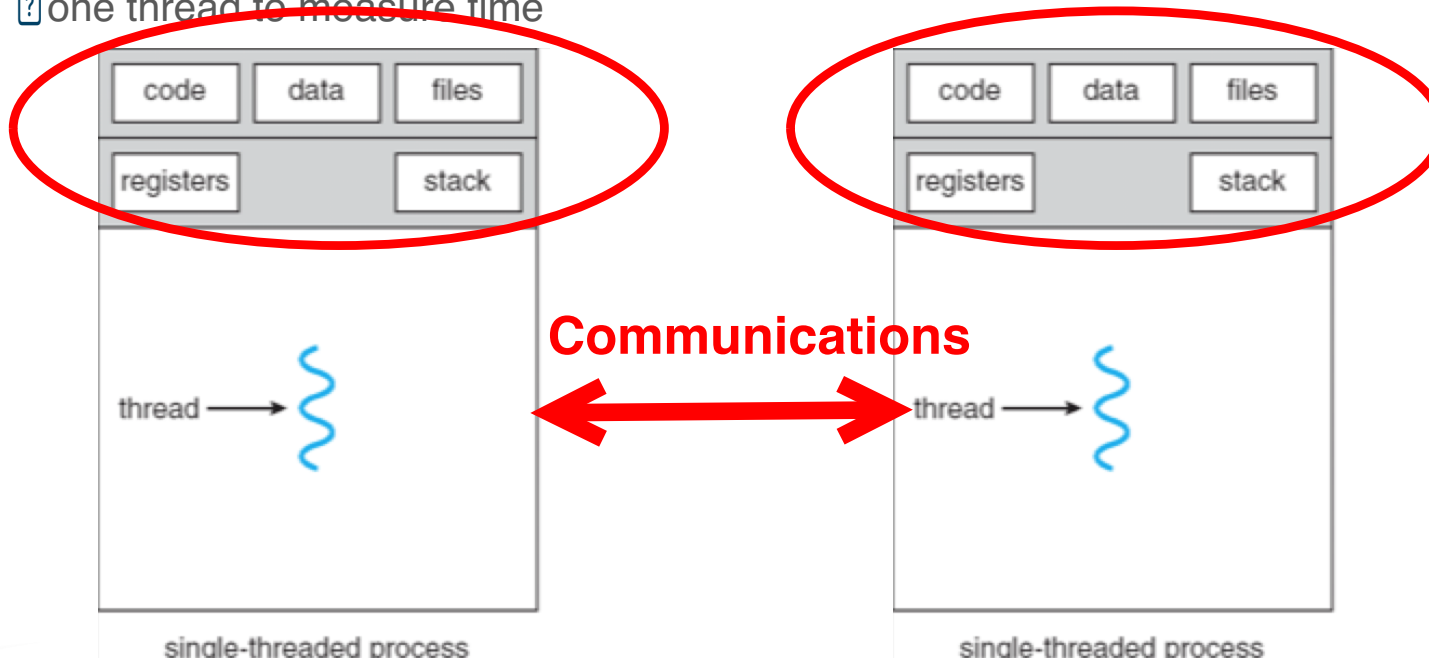
Example: How to Implement This Simple Program?

? The program must:

- ? prompt a user to answer a question
- ? Wait for the user to enter the answer for at most 60s
- ? If the user does not enter any response after 60s, print out “Time Out”

? This program would require two threads of execution:

- ? one thread waiting for the user to enter an answer (blocking call)
- ? one thread to measure time



Example: How to Implement This Simple Program? (Cont'd)

? The program must:

? prompt a user to answer a question

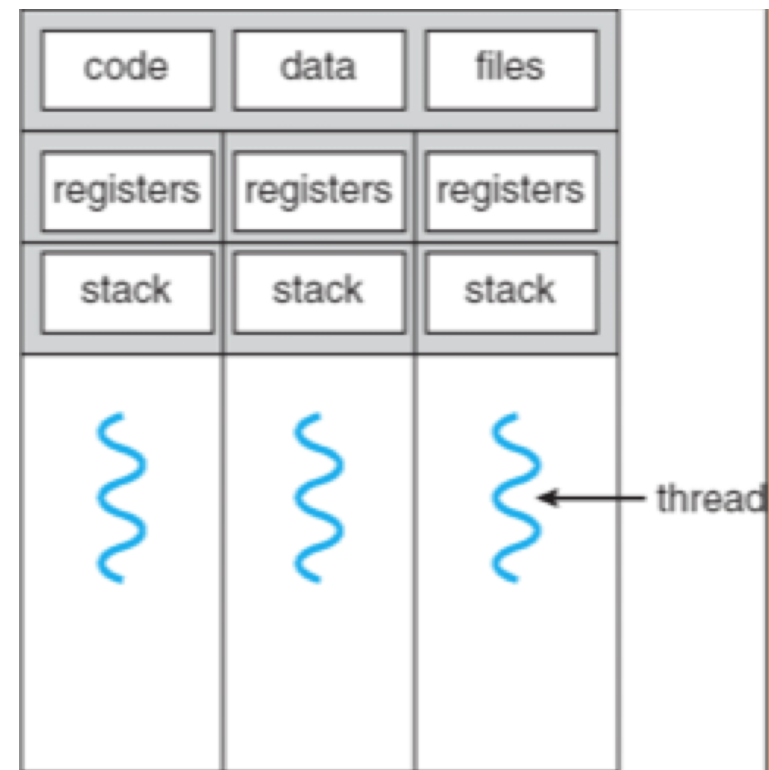
? Wait for the user to enter the answer for at most 60s

? If the user does not enter any response after 60s, print out "Time Out"

? This program would require two threads of execution:

? one thread waiting for the user to enter an answer (blocking call)

? one thread to measure time



Threads Advantages and Drawbacks

? Advantages

- ? Use less memory resources
- ? Offer great flexibility for the programmer

? Drawbacks:

- ? They raise the *data-shared problem*.
- ? This problem arises when a program uses multiple *concurrent* threads *sharing and modifying data*.
- ? **Solution:** This problem will require *process/thread synchronization* to solve this problem. The process/thread synchronization will be covered later.

Wrap Up

- ❑ Learn and understand the concept/notion of a **Process**.
- ❑ Learn and understand how to **represent** a process.
- ❑ Learn and understand **Process Scheduling**.
- ❑ Learn and understand **Threads**.